

Chapter 5: Monte Carlo Methods

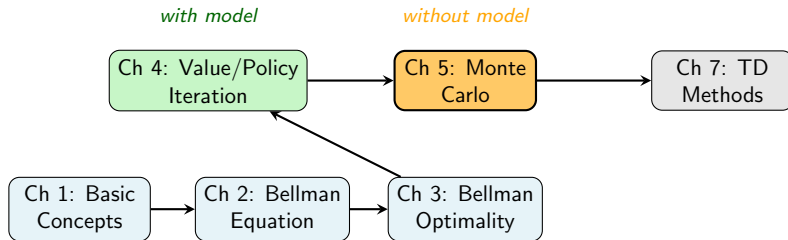
Model-Free Reinforcement Learning

Adapted from Shiyu Zhao's TextBook

Outline

- 1 Introduction
- 2 Mean Estimation: The Motivating Example
- 3 MC Basic: The Simplest Algorithm
- 4 MC Exploring Starts
- 5 MC ϵ -Greedy
- 6 Exploration vs. Exploitation
- 7 Summary

Where We Are in the RL Journey



Key Transition: From model-based to **model-free** algorithms

The Fundamental Philosophy

Model-Free Learning Principle

*"If we do not have a model, we must have some data.
If we do not have data, we must have a model.
If we have neither, then we cannot find optimal policies."*

Model-Based (Chapter 4):

- Requires $p(r|s, a)$ and $p(s'|s, a)$
- Policy Iteration
- Value Iteration

Model-Free (Chapter 5):

- Requires experience samples
- Agent interacts with environment
- Learn from trajectories

Three Monte Carlo Algorithms:

- ① **MC Basic** – The simplest MC algorithm
 - Direct modification of policy iteration
 - Foundation for understanding MC-based RL
- ② **MC Exploring Starts** – More sample-efficient
 - Every-visit strategy
 - Episode-by-episode policy updates
- ③ **MC ϵ -Greedy** – Removes exploring starts requirement
 - Soft policies for exploration
 - Balances exploration and exploitation

Why Mean Estimation Matters

Key Insight

State and action values are defined as **expected values** of returns:

$$v_{\pi}(s) = \mathbb{E}[G_t | S_t = s]$$
$$q_{\pi}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

Estimating state/action values is fundamentally a mean estimation problem!

Mean Estimation: Two Approaches

Problem: Calculate $\mathbb{E}[X]$ for a random variable X with values in set $\mathcal{X}$

Approach 1: Model-Based

If probability distribution $p(x)$ is **known**:

$$\mathbb{E}[X] = \sum_{x \in \mathcal{X}} p(x) \cdot x$$

Approach 2: Model-Free (MC)

If distribution is **unknown**, use samples $\{x_1, x_2, \dots, x_n\}$:

$$\mathbb{E}[X] \approx \bar{x} = \frac{1}{n} \sum_{j=1}^n x_j$$

As $n \rightarrow \infty$, we have $\bar{x} \rightarrow \mathbb{E}[X]$ (Law of Large Numbers)

Example: Coin Flipping Game

Setup:

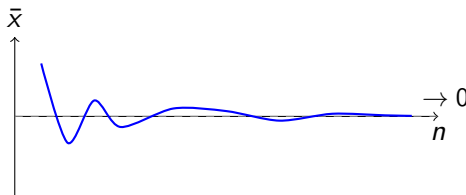
- Random variable X : outcome of coin flip
- $X = +1$ (heads), $X = -1$ (tails)
- True distribution: $p(X = 1) = 0.5$, $p(X = -1) = 0.5$

Model-Based:

$$\begin{aligned}\mathbb{E}[X] &= 0.5 \cdot (+1) + 0.5 \cdot (-1) \\ &= 0\end{aligned}$$

Model-Free:

- Flip coin many times
- Record samples $\{x_i\}_{i=1}^n$
- Compute average $\bar{x}$



Law of Large Numbers

Theorem (Law of Large Numbers)

For i.i.d. samples $\{x_i\}_{i=1}^n$ of random variable X , let $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$. Then:

$$\mathbb{E}[\bar{x}] = \mathbb{E}[X] \quad (\text{unbiased estimate}) \quad (1)$$

$$\text{Var}[\bar{x}] = \frac{1}{n} \text{Var}[X] \quad (\text{variance decreases as } n \rightarrow \infty) \quad (2)$$

Proof of (1):

$$\mathbb{E}[\bar{x}] = \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^n x_i\right] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[x_i] = \frac{1}{n} \cdot n \cdot \mathbb{E}[X] = \mathbb{E}[X]$$

Proof of (2):

$$\text{Var}[\bar{x}] = \text{Var}\left[\frac{1}{n} \sum_{i=1}^n x_i\right] = \frac{1}{n^2} \sum_{i=1}^n \text{Var}[x_i] = \frac{n \cdot \text{Var}[X]}{n^2} = \frac{\text{Var}[X]}{n}$$

Important Requirement: i.i.d. Samples

Critical Assumption

Samples must be **independent and identically distributed (i.i.d.)**

Why does this matter?

- If samples are correlated, mean estimation may fail
- **Extreme case:** All samples equal to the first one
 - Average = first sample (regardless of n)
 - Cannot estimate true mean!

Independence allows variance to decrease
Identical distribution ensures unbiasedness

Converting Policy Iteration to Model-Free

Policy Iteration (Model-Based):

- ① **Policy Evaluation:** Solve $v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}$
- ② **Policy Improvement:** $\pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k})$

Key Observation: Both steps ultimately depend on **action values**:

$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) \cdot q_{\pi_k}(s, a)$$

Question: How can we compute action values without a model?

Two Ways to Compute Action Values

Model-Based Approach

First compute v_{π_k} , then:

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a) \cdot r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s')$$

Requires: $p(r|s, a)$ and $p(s'|s, a)$

Model-Free Approach (MC)

Use definition directly:

$$\begin{aligned} q_{\pi_k}(s, a) &= \mathbb{E}[G_t | S_t = s, A_t = a] \\ &\approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a) \end{aligned}$$

Requires: Episodes starting from (s, a)

MC Key Idea: Replace model-based value computation with sample-based averaging of episode returns

Algorithm 1 MC Basic

```
1: Initialize: Initial policy  $\pi_0$ 
2: for iteration  $k = 0, 1, 2, \dots$  do
3:   for every state  $s \in \mathcal{S}$  do
4:     for every action  $a \in \mathcal{A}(s)$  do
5:       Collect sufficiently many episodes starting from  $(s, a)$  following  $\pi_k$ 
6:       Policy Evaluation:
7:          $q_k(s, a) \leftarrow$  average return of all episodes from  $(s, a)$ 
8:       Policy Improvement:
9:          $a_k^*(s) = \arg \max_a q_k(s, a)$ 
10:         $\pi_{k+1}(a|s) = \begin{cases} 1 & \text{if } a = a_k^* \\ 0 & \text{otherwise} \end{cases}$ 
11:     end for
12:   end for
13: end for
```

MC Basic: Key Properties

Similarities with Policy Iteration:

- Same two-step structure (evaluation + improvement)
- Converges to optimal policy with sufficient samples

Key Differences:

- Directly estimates action values (not state values first)
- No model required – uses experience samples

Why Estimate Action Values Directly?

If we estimated state values $v_\pi(s)$ instead, we would still need the model to compute action values:

$$q_\pi(s, a) = \sum_r p(r|s, a) \cdot r + \gamma \sum_{s'} p(s'|s, a) v_\pi(s')$$

Illustrative Example: Grid World

s_1 ↑	s_2 ↓	s_3 →
s_4	s_5	s_6
s_7	s_8	s_9

Settings:

- $r_{\text{boundary}} = r_{\text{forbidden}} = -1$
- $r_{\text{target}} = +1$
- $\gamma = 0.9$

Initial policy π_0 :

- s_1 : go up (hits boundary)
- Not optimal for s_1 and s_3

Example: Computing Action Values at s_1

Five possible actions at s_1 : a_1 (up), a_2 (right), a_3 (down), a_4 (left), a_5 (stay)

Starting from (s_1, a_1) :

$$s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$$

$$\begin{aligned} q_{\pi_0}(s_1, a_1) &= -1 + \gamma(-1) + \gamma^2(-1) + \dots \\ &= \frac{-1}{1 - \gamma} = -10 \end{aligned}$$

Starting from (s_1, a_2) :

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} \dots$$

$$\begin{aligned} q_{\pi_0}(s_1, a_2) &= 0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3(1) + \dots \\ &= \frac{\gamma^3}{1} = 7.29 \end{aligned}$$

Starting from (s_1, a_4) :

$$s_1 \xrightarrow{a_4} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$$

$$q_{\pi_0}(s_1, a_4) = \frac{-1}{1 - \gamma} = -10$$

Starting from (s_1, a_5) :

$$s_1 \xrightarrow{a_5} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$$

$$q_{\pi_0}(s_1, a_5) = 0 + \gamma(-1) + \dots = \frac{-\gamma}{1 - \gamma}$$

Episode Length and Sparse Rewards

Key Observation: Episode length greatly impacts learning quality

Episode Length	State Values	Notes
1	Only adjacent states $\neq 0$	Too short to reach target
2-4	Partial values estimated	Gradual improvement
14	Nearly optimal policy	Minimum path covered
30+	Optimal values	Sufficient exploration

Sparse Reward Problem

- No positive rewards until reaching target
- Requires long episodes to “discover” rewards
- Solution: Design non-sparse rewards (reward shaping)

Utilizing Samples More Efficiently

Consider an episode:

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$$

Initial-visit strategy (MC Basic): Only estimates $q(s_1, a_2)$

Better idea: Decompose into sub-episodes!

- Original: $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} \dots \rightarrow$ estimate $q(s_1, a_2)$
- Sub-episode: $s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} \dots \rightarrow$ estimate $q(s_2, a_4)$
- Sub-episode: $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} \dots \rightarrow$ estimate $q(s_1, a_2)$
- Sub-episode: $s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \rightarrow$ estimate $q(s_2, a_3)$
- Sub-episode: $s_5 \xrightarrow{a_1} \dots \rightarrow$ estimate $q(s_5, a_1)$

Visit Strategies

First-Visit Strategy

Count only the **first time** each (s, a) pair appears in an episode

Example: (s_1, a_2) appears twice
→ Only use first occurrence

Every-Visit Strategy

Count **every time** each (s, a) pair appears in an episode

Example: (s_1, a_2) appears twice
→ Use both occurrences

Sample Efficiency: Every-visit $>$ First-visit $>$ Initial-visit

Note on Correlation

Every-visit samples are correlated (sub-episodes overlap), but correlation is weak if visits are far apart in the trajectory.

When to Update Policies?

Strategy 1: Batch Updates (MC Basic)

- Collect *all* episodes for each (s, a)
- Compute average return
- Then update policy
- **Drawback:** Must wait for all episodes

Strategy 2: Episode-by-Episode Updates

- Use single episode's return as rough estimate
- Update policy immediately
- **Advantage:** Faster learning
- Falls under **Generalized Policy Iteration** framework

Policy can be updated even with inaccurate value estimates!

MC Exploring Starts Algorithm

Algorithm 2 MC Exploring Starts

```
1: Initialize:  $\pi_0(a|s)$ ,  $q(s, a)$ ,  $\text{Returns}(s, a) = 0$ ,  $\text{Num}(s, a) = 0$  for all  $(s, a)$ 
2: for each episode do
3:   Select starting pair  $(s_0, a_0)$  ensuring all pairs can be selected (exploring starts)
4:   Generate episode:  $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 
5:    $g \leftarrow 0$ 
6:   for  $t = T - 1, T - 2, \dots, 0$  do
7:      $g \leftarrow \gamma g + r_{t+1}$ 
8:      $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$ 
9:      $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$ 
10:    Policy Evaluation:  $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$ 
11:    Policy Improvement:  $\pi(a|s_t) = \mathbf{1}[a = \arg \max_{a'} q(s_t, a')]$ 
12:  end for
13: end for
```

- Backward iteration for efficient return computation
- Every-visit strategy
- Episode-by-episode updates

Backward Return Computation

Why iterate backward?

Naive approach: For each time t , compute

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots + \gamma^{T-t-1} r_T$$

Complexity: $O(T^2)$ for all returns

Efficient backward iteration:

$$G_{T-1} = r_T$$

$$G_{T-2} = r_{T-1} + \gamma G_{T-1}$$

$$G_{T-3} = r_{T-2} + \gamma G_{T-2}$$

$$\vdots$$

$$G_t = r_{t+1} + \gamma G_{t+1}$$

Complexity: $O(T)$ for all returns!

The Exploring Starts Condition

Exploring Starts Requirement

Sufficiently many episodes must start from **every** state-action pair (s, a)

Why is this necessary?

- Law of Large Numbers requires many samples
- Under-explored actions may have inaccurate values
- May miss optimal actions if not explored enough

Practical Challenge

In many applications (especially with physical interactions), we **cannot** freely choose starting states!

Solution: Use ϵ -greedy policies (next section)

Definition: Soft Policy

A policy is **soft** if it has a positive probability of taking **any** action at **any** state:

$$\pi(a|s) > 0 \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A}(s)$$

Why soft policies help:

- A single sufficiently long episode can visit every (s, a) pair
- No need for exploring starts!
- All actions get explored naturally

ϵ -Greedy Policies

Definition: ϵ -Greedy Policy

For $\epsilon \in [0, 1]$ and greedy action $a^* = \arg \max_a q(s, a)$:

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) & \text{if } a = a^* \text{ (greedy)} \\ \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a \neq a^* \text{ (non-greedy)} \end{cases}$$

Special cases:

- $\epsilon = 0$: Pure greedy policy
- $\epsilon = 1$: Uniform random policy (all actions equally likely: $\frac{1}{|\mathcal{A}(s)|}$)

Property: Greedy action always has highest probability:

$$1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} \geq \frac{\epsilon}{|\mathcal{A}(s)|}$$

Implementing ϵ -Greedy Action Selection

Algorithm 3 ϵ -Greedy Action Selection

- 1: Generate random number $x \sim \text{Uniform}(0, 1)$
 - 2: **if** $x \geq \epsilon$ **then**
 - 3: Select greedy action: $a = a^* = \arg \max_a q(s, a)$
 - 4: **else**
 - 5: Select random action: $a \sim \text{Uniform}(\mathcal{A}(s))$
 - 6: **end if**
-

Resulting probabilities:

- Greedy action: $(1 - \epsilon) + \frac{\epsilon}{|\mathcal{A}(s)|} = 1 - \frac{\epsilon(|\mathcal{A}(s)| - 1)}{|\mathcal{A}(s)|}$
- Each non-greedy action: $\frac{\epsilon}{|\mathcal{A}(s)|}$

MC ϵ -Greedy: Policy Improvement Step

Original (Greedy):

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi} \sum_a \pi(a|s) q_{\pi_k}(s, a)$$

where Π = all possible policies

ϵ -Greedy Version:

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi_\epsilon} \sum_a \pi(a|s) q_{\pi_k}(s, a)$$

where Π_ϵ = all ϵ -greedy policies

Solution:

$$\pi_{k+1}(a|s) = \begin{cases} 1 - \frac{(|\mathcal{A}(s)|-1)\epsilon}{|\mathcal{A}(s)|} & \text{if } a = a_k^* \\ \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a \neq a_k^* \end{cases}$$

where $a_k^* = \arg \max_a q_{\pi_k}(s, a)$

MC ϵ -Greedy Algorithm

Algorithm 4 MC ϵ -Greedy

```
1: Initialize:  $\pi_0(a|s)$ ,  $q(s, a)$ ,  $\text{Returns}(s, a) = 0$ ,  $\text{Num}(s, a) = 0$ ,  $\epsilon \in (0, 1]$ 
2: for each episode do
3:   Select starting  $(s_0, a_0)$  (exploring starts NOT required)
4:   Generate episode following current policy:  $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 
5:    $g \leftarrow 0$ 
6:   for  $t = T - 1, T - 2, \dots, 0$  do
7:      $g \leftarrow \gamma g + r_{t+1}$ 
8:      $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$ 
9:      $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$ 
10:    Policy Evaluation:  $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$ 
11:    Policy Improvement: Let  $a^* = \arg \max_a q(s_t, a)$ 
12:      
$$\pi(a|s_t) = \begin{cases} 1 - \frac{(|\mathcal{A}(s_t)|-1)\epsilon}{|\mathcal{A}(s_t)|} & \text{if } a = a^* \\ \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{otherwise} \end{cases}$$

13:    end for
14: end for
```

Optimality of ϵ -Greedy Policies

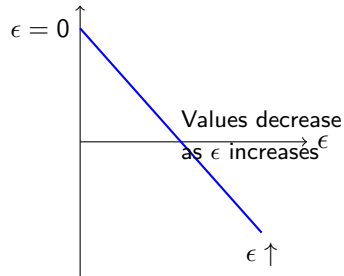
Good News:

- With sufficient samples, converges to optimal ϵ -greedy policy in Π_ϵ

Bad News:

- Optimal in Π_ϵ may not be optimal in Π
- Suboptimality increases with ϵ

State Value



Practical Advice

If ϵ is **sufficiently small**, optimal ϵ -greedy policies are close to truly optimal greedy policies.

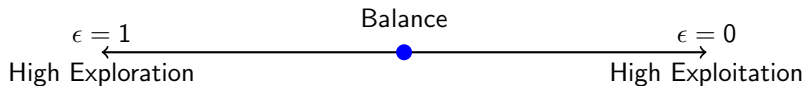
The Fundamental Tradeoff

Exploration

- Try different actions
- Discover better options
- Improve value estimates
- Risk: Wasting time on bad actions

Exploitation

- Use current best action
- Maximize immediate reward
- Follow greedy policy
- Risk: Missing better options



Effect of ϵ on State Values

Same greedy action structure, different ϵ values:

ϵ	Target State Value	Other States	Interpretation
0	10.0	Positive	Optimal
0.1	3.4	Lower	Near-optimal
0.2	-2.5	Negative	Exploration hurts
0.5	-17.0	Very negative	Too much exploration

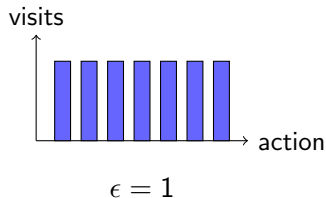
Why do values decrease with ϵ ?

- Higher $\epsilon \rightarrow$ more random actions
- Agent may enter forbidden/penalty areas
- Even at target, may “accidentally” leave

Exploration Ability: $\epsilon = 1$ vs $\epsilon = 0.5$

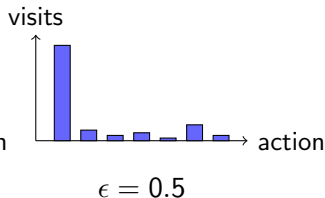
$\epsilon = 1$ (Uniform Random):

- Equal probability for all actions
- Visits all (s, a) pairs evenly
- After 1M steps: each action visited ~ 8000 times
- Strong exploration



$\epsilon = 0.5$:

- Prefers greedy actions
- Uneven visitation pattern
- After 1M steps: some actions visited 250,000+ times, others only tens
- Weaker exploration



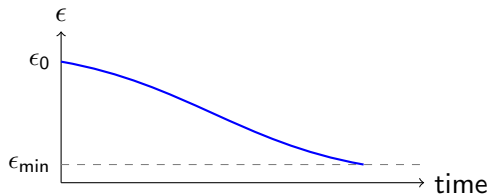
Practical Technique: ϵ -Decay

ϵ -Decay Strategy

- Start with large ϵ (more exploration)
- Gradually reduce ϵ over time
- End with small ϵ (more exploitation)

Common decay schedules:

- Linear: $\epsilon_t = \epsilon_0 - \frac{t}{T}(\epsilon_0 - \epsilon_{\min})$
- Exponential: $\epsilon_t = \epsilon_0 \cdot \alpha^t$ for $\alpha < 1$



Summary: The Tradeoff

	Large ϵ	Small ϵ
Exploration	Strong	Weak
Exploitation	Weak	Strong
Value Estimates	Better coverage	More focused
Optimality	Suboptimal	Near-optimal
Coverage	All (s, a) visited	May miss some

Key Insight

ϵ -greedy policies balance exploration and exploitation by:

- Higher probability for greedy action (exploitation)
- Non-zero probability for all other actions (exploration)

Three MC Algorithms:

① MC Basic

- Simplest MC algorithm
- Model-based policy evaluation $\rightarrow$ MC estimation
- Requires exploring starts

② MC Exploring Starts

- Every-visit strategy for efficiency
- Episode-by-episode updates
- Still requires exploring starts

③ MC ϵ -Greedy

- Uses soft (ϵ -greedy) policies
- Removes exploring starts requirement
- Balances exploration/exploitation

Key Concepts Recap

Mean Estimation:

- $\mathbb{E}[X] \approx \frac{1}{n} \sum_{i=1}^n x_i$
- Law of Large Numbers
- Requires i.i.d. samples

Visit Strategies:

- Initial-visit
- First-visit
- Every-visit

Exploring Starts:

- Visit all (s, a) sufficiently
- Necessary for convergence
- Often impractical

ϵ -Greedy:

- Soft policy
- Exploration vs exploitation
- ϵ -decay technique

Algorithm Comparison

Feature	MC Basic	MC ES	MC ϵ -Greedy
Visit Strategy	Initial	Every	Every
Update Timing	Batch	Per-episode	Per-episode
Exploring Starts	Required	Required	Not required
Policy Type	Greedy	Greedy	ϵ -greedy
Sample Efficiency	Low	Medium	Medium
Convergence	Optimal	Optimal	ϵ -optimal

Next: Temporal-Difference Methods (Chapter 7)
Learn from incomplete episodes!

Q: Why estimate action values directly instead of state values?

A: To avoid needing the model for $q(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v(s')$

Q: Can one episode visit all state-action pairs?

A: Yes, if the policy is soft (e.g., ϵ -greedy) and episode is sufficiently long.

Q: Can ϵ -greedy policies be optimal?

A: Yes in Π_ϵ , but generally suboptimal in full policy space Π .

Q: Why is exploring starts impractical?

A: Cannot always choose starting states in real environments (e.g., robotics).

Q: How to handle sparse rewards?

A: Use longer episodes, reward shaping, or non-sparse reward design.

Thank You!

Questions?

Next: Chapter 6 (Stochastic Approximation) & Chapter 7 (TD Methods)